

Set up Selenium

Week 8 · before Wednesday's lab · about 15 minutes

Brian C. Keegan, Ph.D. · Department of Information Science, University of Colorado Boulder

October 5, 2026

What you do

You run four cells from chapter 8 in Jupyter, one at a time, and check what each one prints before you run the next. The fourth opens Chrome. Then a cell of your own visits a page, prints its title, and closes Chrome again.

The cells are in chapter 8's section **Before the First Browser**. Copy each one with the **Copy to Clipboard** button at its top right, or open the chapter's companion notebook, which the top of the chapter links to.

The first run downloads files: the driver, about 10 MB, and, if your computer has no Chrome, Chrome for Testing, about 190 MB. Do it on a fast connection, not in the minutes before the lab.

Selenium Manager

A small program that comes with Selenium. It finds your Chrome, downloads the driver that matches it, and downloads Chrome for Testing if you have no Chrome. It keeps them in `~/.cache/selenium`.

No conda activate needed

Step 1 finds Selenium Manager through your notebook's own Python, however you started Jupyter.

1 · Settings first

Start Jupyter the way you do for every lab, and open a new notebook. Paste step 1's cell into it and run it with Shift+Enter. It prints two lines (Figure 1):

- 1 Selenium's version. Steps 2 and 3 need 4.20 or later.
- 2 Where Selenium Manager is: in webdata's own folder, under bin on macOS and Linux, or Scripts on Windows. The cell looked there and saved the path in `SE_MANAGER_PATH`, the setting that conda activate makes. If this line says inside the selenium package, your Selenium came from pip, and that works too.

No module named 'selenium'?

Install it from a terminal (on Windows, the Anaconda Prompt). `-n webdata` installs into webdata without activating it:

```
conda install -n webdata -c conda-forge selenium
```

Type y when conda asks. Then, in Jupyter, choose **Kernel** → **Restart Kernel...** → **Restart**, and run step 1 again. Restart the kernel, not Jupyter: step 1 sets what conda activate would have set. For a version older than 4.20, install "selenium>=4.20" the same way.

```
[1]: import os
import sys
from pathlib import Path

import selenium

print("selenium", selenium.__version__) # the next cell needs 4.20 or later

# Where conda-forge puts Selenium Manager. Activating webdata sets SE_MANAGER_PATH to it
manager = Path(sys.prefix) / ("Scripts/selenium-manager.exe" if os.name == "nt" else "bin/selenium-manager")
if "SE_MANAGER_PATH" not in os.environ and manager.is_file():
    os.environ["SE_MANAGER_PATH"] = str(manager)
print("Selenium Manager:", os.environ.get("SE_MANAGER_PATH", "inside the selenium package"))

os.environ["SE_SKIP_DRIVER_IN_PATH"] = "true" # ignore stray drivers on your PATH
os.environ["SE_AVOID_STATS"] = "true" # send no usage statistics (optional)
# os.environ["SE_PROXY"] = "http://proxy.example.edu:3128" # only behind a proxy

selenium 4.50.0 1
Selenium Manager: /opt/miniforge3/envs/webdata/bin/selenium-manager 2
```

Figure 1. Step 1 after it ran: Selenium 4.50.0 1, and Selenium Manager in webdata's bin folder 2. Two long lines run past the cell's right edge, as long lines do in Jupyter.

Its last three lines are settings for the steps after it: ignore old drivers on your PATH, send no usage statistics, and, commented out, a proxy for a network that needs one.

2 · Run Selenium Manager

Paste step 2's cell into the next cell and run it. It asks Selenium Manager which driver and which browser webdriver.Chrome() will use, and shows Selenium Manager's log. The first time, the cell can take a minute or more while it downloads.

The log is pink because it goes to Python's error stream; pink here doesn't mean that something failed. Read it from the top (Figure 2):

- ① chrome not found in the system: this computer has no Chrome. If yours has Chrome, the log says Detected browser: chrome and its version here, and ② doesn't happen.
- ② It downloads Chrome for Testing, a build of Chrome made for automation.
- ③ It downloads the chromedriver made for that version of Chrome.
- ④ Where it keeps the two: in ~/.cache/selenium, or %USERPROFILE%\cache\selenium on Windows. The next run finds them there, and nothing downloads.

```
[2]: import logging

from selenium.webdriver.common.selenium_manager import SeleniumManager

logging.basicConfig(level=logging.WARNING, format="%(levelname)s %(message)s")
log = logging.getLogger("selenium.webdriver.common.selenium_manager")

log.setLevel(logging.DEBUG) # show each decision Selenium Manager makes
paths = SeleniumManager().binary_paths(["--browser", "chrome"])
log.setLevel(logging.WARNING) # from here on, show only its warnings

DEBUG Selenium Manager set by env SE_MANAGER_PATH to: /opt/miniforge3/envs/webdata/bin/selenium-manager
DEBUG Selenium Manager binary found at: /opt/miniforge3/envs/webdata/bin/selenium-manager
DEBUG Executing process: /opt/miniforge3/envs/webdata/bin/selenium-manager --browser chrome --debug --language-binding python --output json
DEBUG chromedriver not found in PATH
DEBUG chrome not found in PATH
① DEBUG chrome not found in the system
DEBUG Discovering versions from https://googlechromelabs.github.io/chrome-for-testing/latest-known-good-versions-with-downloads.json
DEBUG Required browser: chrome 154.0.8037.92
DEBUG Acquiring lock: /home/student/.cache/selenium/chrome/linux64/154.0.8037.92/sm.lock
② DEBUG Downloading chrome 154.0.8037.92 from https://storage.googleapis.com/chrome-for-testing-public/154.0.8037.92/linux64/chrome-linux64.zip
DEBUG chrome 154.0.8037.92 is available at /home/student/.cache/selenium/chrome/linux64/154.0.8037.92/chrome
DEBUG Discovering versions from https://googlechromelabs.github.io/chrome-for-testing/known-good-versions-with-downloads.json
DEBUG Required driver: chromedriver 154.0.8037.92
DEBUG Acquiring lock: /home/student/.cache/selenium/chromedriver/linux64/154.0.8037.92/sm.lock
③ DEBUG Downloading chromedriver 154.0.8037.92 from https://storage.googleapis.com/chrome-for-testing-public/154.0.8037.92/linux64/chromedriver-linux64.zip
DEBUG Driver path: /home/student/.cache/selenium/chromedriver/linux64/154.0.8037.92/chromedriver
④ DEBUG Browser path: /home/student/.cache/selenium/chrome/linux64/154.0.8037.92/chrome
```

Figure 2. Step 2's first run on a computer with no Chrome and an empty cache (October 5, 2026): no Chrome ①, so Selenium Manager downloads Chrome for Testing ② and its driver ③, and keeps both in its cache ④.

Check that no line starts with WARNING. Selenium Manager warns instead of stopping, and a warning now can become an error when the browser starts.

3 · Check before you start the browser

Paste step 3's cell and run it. Start the browser only when its output looks like Figure 3:

- ① Both paths end in (found).
- ② The last line says True: the driver is the one that Selenium Manager matched to your browser, not an old one from your PATH.

```
[3]: from pathlib import Path

driver_path = Path(paths["driver_path"])
browser_path = Path(paths["browser_path"])
cache = Path.home() / ".cache" / "selenium"

print("Driver: ", driver_path, "(found)" if driver_path.is_file() else "(MISSING)")
print("Browser:", browser_path, "(found)" if browser_path.is_file() else "(MISSING)")
print("Driver chosen by Selenium Manager:", cache in driver_path.parents)

1 { Driver: /home/student/.cache/selenium/chromedriver/linux64/154.0.8037.92/chromedriver
   (found)
2 { Browser: /home/student/.cache/selenium/chrome/linux64/154.0.8037.92/chrome (found)
   Driver chosen by Selenium Manager: True
```

Figure 3. Step 3 after steps 1 and 2: both paths found ①, and the driver is Selenium Manager's choice ②.

4 • Start the browser

1. Paste the cell under **Starting the Browser** (it holds `driver = webdriver.Chrome()`) and run it. A Chrome window opens (Figure 4). Leave it alone: your code drives it.
2. In a new cell, visit a page, print its title, and close the browser:

```
driver.get("https://quotes.toscrape.com/")
print(driver.title)
driver.quit()
```

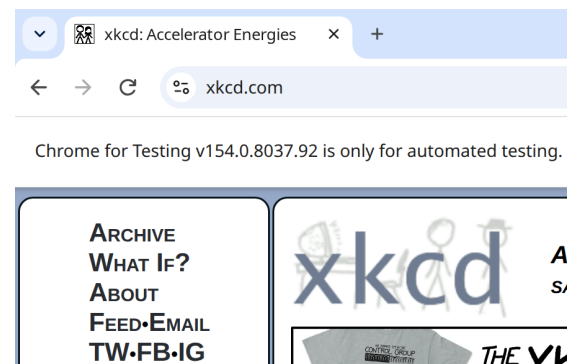


Figure 4. A window that `webdriver.Chrome()` opened, here on `xkcd.com`. Chrome for Testing says that it is only for automated testing; Google Chrome says that it is being controlled by automated test software.

```
[4]: from selenium import webdriver

driver = webdriver.Chrome()

# The browser that opened, and the driver controlling it
print("Browser:", driver.capabilities["browserVersion"])
print("Driver: ", driver.capabilities["chrome"]["chromedriverVersion"].split()[0])

Browser: 154.0.8037.92
Driver: 154.0.8037.92 ①

[5]: driver.get("https://quotes.toscrape.com/")
print(driver.title)
driver.quit()

Quotes to Scrape ②
```

Figure 5. The browser cell and the visit cell: the browser's and the driver's versions ①, and the page's title ②, printed before `driver.quit()` closed the window.

The two versions ① should match up to the first dot. End every session with `driver.quit()`: each driver is a whole browser, and one you forget keeps running and using memory.

If something goes wrong

NoSuchDriverException: Message: Unable to obtain driver for chrome at the bottom of a long red traceback, and **Unable to obtain working Selenium Manager binary** partway up it (Figure 6). You started the browser before step 1, in a Jupyter that started without `webdata` active or before you installed Selenium, so Selenium couldn't find Selenium Manager. Run step 1, then the browser cell again.

```
WebDriverException: Message: Unable to obtain working Selenium Manager binary; /opt/miniforge3/envs/webdata/lib/python3.14/site-packages/selenium/webdriver/common/linux-x86_64/selenium-manager

The above exception was the direct cause of the following exception:

Partway down the traceback

76 except Exception as err:
77     msg = f"Unable to obtain driver for {browser}"
--> 78     raise NoSuchDriverException(msg) from err
79 return self._paths

NoSuchDriverException: Message: Unable to obtain driver for chrome; For documentation on this error, please visit: https://www.selenium.dev/documentation/webdriver/troubleshooting/errors/driver_location

At the bottom
```

Figure 6. The browser cell run without step 1, in a Jupyter started without `SE_MANAGER_PATH`: the cause, partway down the traceback, and the message at its bottom.

- **This version of ChromeDriver only supports Chrome version *N*:** an old chromedriver on your `PATH` won. Step 1 tells Selenium Manager to skip it: run steps 1 to 3, then the browser cell.
- **The downloads in step 2 fail:** your network may block Google's download servers. Run steps 1 and 2 once on another network, such as your home Wi-Fi; after that, Selenium Manager works from its cache.
- **Ubuntu 24.04 or later, and session not created: Chrome instance exited:** Ubuntu doesn't let Chrome for Testing use its sandbox. Install Google Chrome from <https://www.google.com/chrome/>, and run steps 2 to 4 again; Selenium Manager then finds it. Don't add `--no-sandbox`.
- **Something else:** bring the whole message, from its first red line to its last, to Wednesday's lab. Chapter 8's "When Selenium Manager Fails" lists more causes.